Mining Software Repositories

Chapter 3: Enriching Raw Data

Prof. Dr. Steffen Herbold

Contents

- Measurements and Software Metrics
- Links between repositories
- Collecting defect data
- Noise in MSR data
- Coding of data

Reasons for Enrichment

- Raw data is collected individually for each data source

- Often no processing of data during the collection
  - No linking between sources
  - No calculation of indirect information
  - No tagging

- Often not feasible during the collection
  - May require full data from one or multiple sources

  → By definition only possible after collection

- Enrichment is an important step to prepare data for the subsequent analysis
  - Does not fit into any specific category within a normal data science workflow (e.g., pre-processing, data collection, feature engineering), rather a mixture that can happen in different phases
- Enrichment usually driven by purpose, i.e., goals of a study

Measurements and Software Metrics

Defining Measurement

Measurement is the process by which numbers or symbols are assigned to attributes of entities […] in such a way as to describe them accordingly to clearly defined rules.
(Fenton, Pfleeger. Software Metrics – A Rigouros & Practical Approach)

Measurement is the empirical, objective assignment of numbers, according to a derived rule from a model or theory, to attributes of objects or events with the intent of describing them.
(Craner, Bond. Software Engineering Metrics: What Do They Measure and How Do We Know?)

A quantitatively expressed reduction of uncertainty based on one or more observations.
(Hubbard. How to Measure Anything: finding the value of intangibles in business)

You cannot control what you cannot measure.
(DeMarco. Controlling Software Projects, Management Measurement & Estimation

Measurement terminology

- *Quantification* is turning observations into numbers

- *Measure* refers to a method or standard format for measuring something
  - According to ISO/IEC/IEEE 24765:2017:
    - variable to which a value is assigned as the result of measurement

  - A measure m is defined as a map $m: O \rightarrow V$ from the object space in which the objects we measure reside O into some space of measurement values V
  - V can be defined on different scales, e.g., the NOIR scales
    - Nominal, i.e., categories without order
    - Ordinal, i.e., categories with order
    - Interval, i.e., numeric scale where differences are well defined but where ratios are not reasonable (e.g., dates)
    - Ratio, i.e., numeric scales in which ratios are also well defined

- *Operationalization* is identifying and implementing a method to measure some factors

Direct and indirect measures

- Direct measure directly quantify the attributes of interest
  - Examples: costs, time, size
- Indirect measures (also called pseudo-measures, proxies) measure something related to the attribute of interest
  - Used when there is no direct measure or the direct measurement is too expensive
  - Use other directly measurable aspects instead
    - These aspects severe as proxy through which the desired property is measured
    - Example:
      - Cost of development cannot be directly quantified because required data about time invested and personal costs is missing
      - Measure size of produced code as proxy for costs, assuming that size (e.g., lines of code) is a suitable proxy for costs
  - The proxies must be correlated to the attribute of interest
    - Strong correlation means we have a good indirect measure
      - On a large scale, aspects like complexity are often strongly correlated to size
    - Weak correlation means that our indirect measure my be problematic
      - E.g., difficult and complex code can be expensive, but small
  - Indirect measures can also be defined through a combination of other direct measures
    - Example:
      - Measure productivity as the ratio of created lines of code and time

Types of measures

- Objective measure: quantification based on objective facts
  - E.g., counting the non-empty lines of code
- Subjective measure: quantification may be subjective, e.g., due to estimations or human judgement
  - E.g., measure how readable code is

- Diagnostic measure: measure current state
- Prognostic measure: measure how the state will be

- Robust measure: measure cannot be influence by humans (or other adversarial aspects)
- Sensitive measure: measure can be influenced to achieve desired outcomes

Quality attributes of measures (1/2)

- Plausibility / validity
  - Does the measure actually measure the desired property?
  - This follows from the representational theory of measurement, which means that scales must be mappable to our empirical experiences
- Relevance
  - Is the measure useful and can it be interpreted in a meaningful way?
- Cost efficiency
  - How much does a measurement cost?
  - Are the costs warranted given the relevance?
- Analyzability
  - Is the measure suitable for further downstream analysis?
  - What needs to be considered?
  - Related to metric scales, but also to distributions, etc.

Quality attributes of measures (2/2)

- Availability
  - Can we measure an attribute at the time we need the measurement?
- Resolution
  - Can the measure capture meaningful and noticeable differences?
- Reproducibility
  - Do we get the same results if we apply the measure multiple times or in different environments?
  - E.g., different people, different tools, different subjects

Software metrics

- Metrics according to ISO/IEC/IEEE 24765:2017:
  - Quantitative measure of the degree to which a system, component, or process possesses a given attribute
  - Defined measurement method and the measurement scale

- Software quality metrics according to ISO/IEC/IEEE 24765:2017:
  - function whose inputs are software data and whose output is a single numerical value that can be interpreted as the degree to which software possesses a given attribute that affects its quality.

→ Metrics are the operationalization of measures!

Types of software metrics

- Types of metrics and attributes after ISO/IEC/IEEE 24765:2017:
  - Product metrics
    - metric used to measure the characteristics of any intermediate or final product of the software development process
  - Process metrics
    - metric used to measure characteristics of the methods, techniques, and tools employed in developing, implementing, and maintaining the software system
  - Internal attributes
    - measurable property of an entity which can be derived purely in terms of the entity itself
  - External attributes
    - measurable property of an entity which can only be derived with respect to how it relates to its environment
- There are also other classifications in the literature that, e.g., consider resources as well, but we follow the standards that only use these two types

Classification of software metrics

- Metric types and attribute are orthogonal and can be combined

	Product metric	**Process metric**
Internal attribute	**Internal product metric** that measures, e.g., size or complexity	**Internal process metric** that measures, e.g., durations or effort
External attribute	**External product metric** that measures, e.g., testability or interoperability	**External process metric** that measures, e.g., costs or number of bugs

Examples for internal product metrics

- Measuring the size in *Source Lines Of Code (SLOC)*
  - Lines of code without empty lines and comments
  - There are many variants of LOC, e.g., Comment LOC (CLOC), Test LOC (TLOC)
- Measuring the complexity through the *Cyclomatic Complexity (CC)*
  - Also known as McCabe‘s Complexity
  - Measures the number of branches in code
- Object-oriented metrics like the *Chidamber & Kemerer Metrics* suite
  - Popular metric suite defined with six metrics to measure aspects of object-oriented software proposed in the 90s
  - E.g., Weighted Method per Class (WMC, sum of CC of all methods in a class) and Response For a Class (RFC, number of objects that a class calls)
- Measuring self-admitted technical debt by counting TODO (and similar) code comments.
  - Technical debt is a metaphor that means that (internal) code quality decreases over time, if not actively improved
  - Self-admitted technical debt means that developers directly admit the debt during development, e.g., through TODO comments.
- …

→ Internal product metrics are often related to how maintainable a software product is

Internal product metrics often measured directly based on source code.
Measurement can be part of raw data collection with static analysis tools!

Examples for external product metrics

- Measuring the performance through the *Mean Time To Response (MTRR)*
  - Quantifies how long a user needs to wait for a response, e.g., of a Web service
  - Can be inferred by analyzing log data

- Measuring the reliability through the *Mean Time Between Failures (MTBF)*
  - Quantifies how much time passes between two outages of a system
  - Can be inferred by analyzing log data

- Measuring the *defect density*, i.e., the ratio of number of bugs and size
  - Quantifies the external quality of the code
  - Can be inferred by analyzing commit messages and/or issues for bugs and code for size
- …

→ External product metrics are often closely related to how users perceive software quality

Examples for internal process metrics

- Measure the *development velocity*
  - Used by SCRUM to estimate how much work can be done within in sprint
  - Usually measured by the number of story points completed within a sprint
  - Can be inferred from the issue tracker

- Measure the *Mean Time to Resolve*
  - Important metric to track if the development can keep up with the amount of reported issues
  - Can be defined in two variants, i.e., for all issues or only for bugs
  - Can be inferred from the issue tracker

- Measure the *Code Review Coverage*
  - The amount of code review conducted is an indicator for the adherence to quality assurance best practices
  - Can be inferred from code review systems

→ Internal process metrics are often related to measuring progress, speed, or adherence to best practices

Examples for external process metrics

- Measure if development is on track through the deviation from schedule
  - Can be inferred if the project plan is available within the repository
  - Can also use data from issue tracking to see if issues were deferred to later releases

- Measure how fast reactions of the support are through the Customer Support Response Time
  - Can be inferred from issue tracking data
  - Not to be mixed up with the internal aspect of actually addressing concerns, just considers the time to respond
    - Response could also be that something will be addressed at a later time

- Post-release defects
  - Defects after the release should be prevented by the development process, e.g., through testing and code review
  - Many defects are an indicator that the process has a low quality with respect to external attributes
  - Can be inferred from issue tracking data

→ External process metrics are often related to quality issues related to costs or processes affecting users

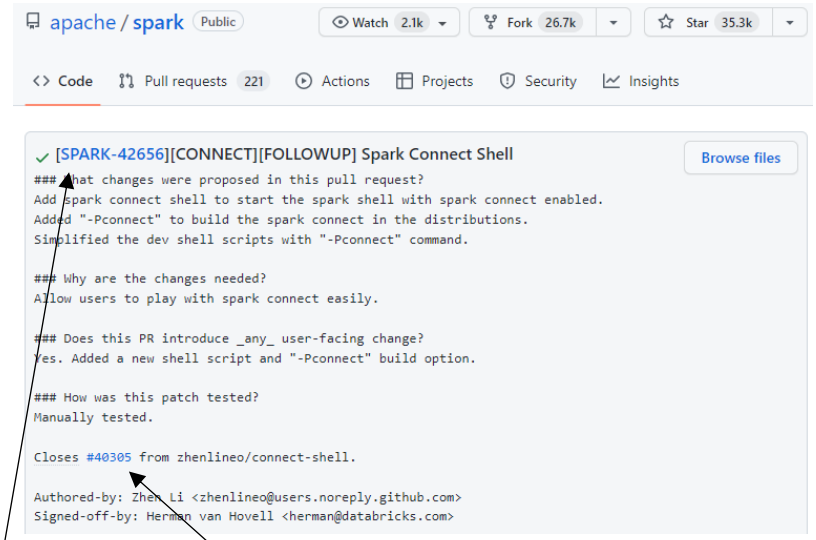
MSR to operationalize measures

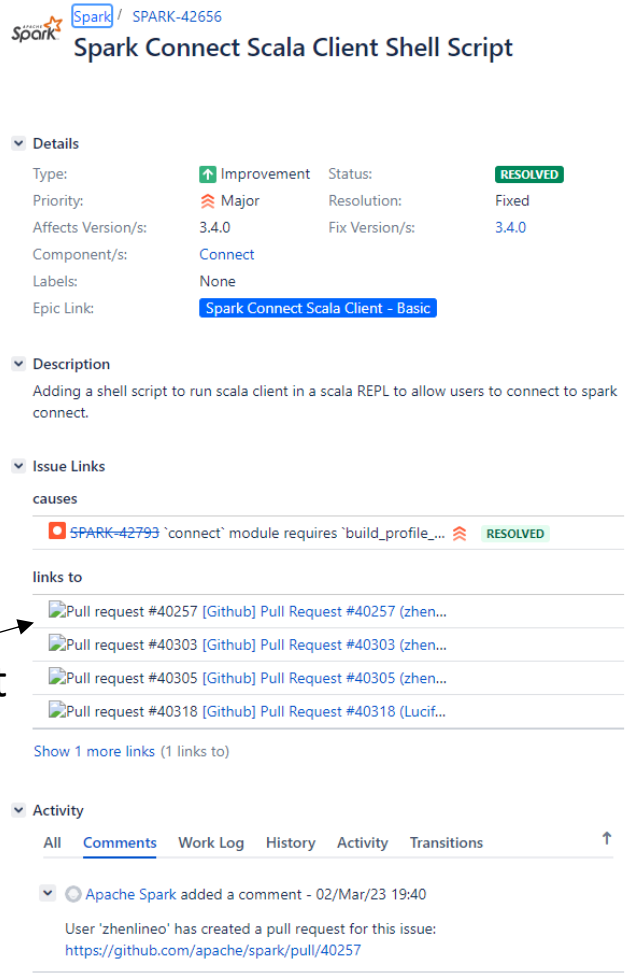
- At its core, MSR provides data about software engineering
- The data can be used as foundation for measurements
- MSR suitable to obtain measurements for all four kinds of software metrics
  - E.g., code quality based on collected source code
  - E.g., mean time to fix a bug based on issue tracking data
  - Number of bugs over time based on issue tracking data

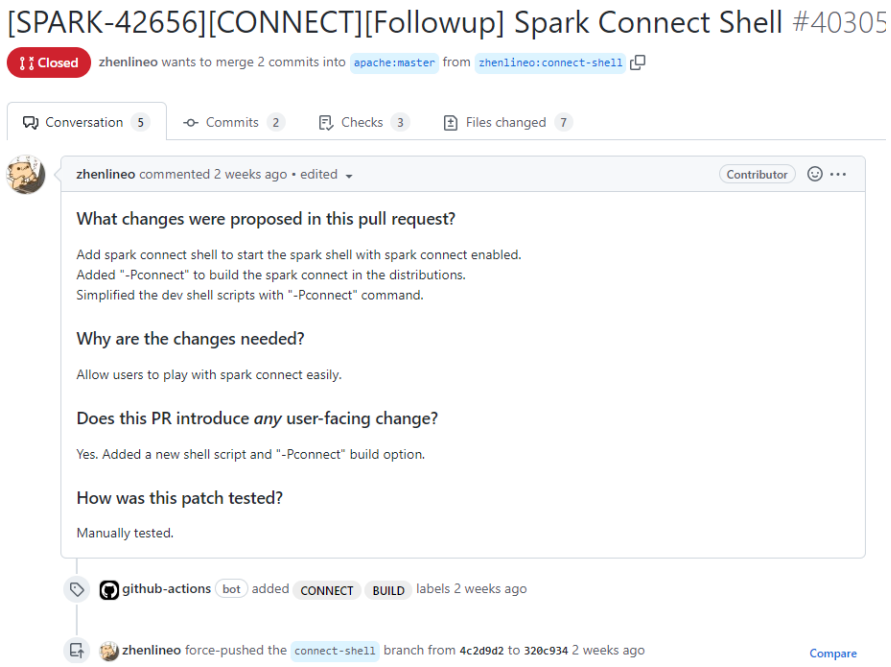
→ Collection of software metrics about raw data enriches data!

Links between repositories

Examples for links







Commit → Issue

Commit → Pull Request

Issue → Pull Request

Pull Request → Commit

https://github.com/apache/spark/commit/2e7207f96e1ff848def135de63f63bcda7402517 https://issues.apache.org/jira/browse/SPARK-42656 https://github.com/apache/spark/pull/40305

Cross-repository analysis

- Many aspects of software development leave traces in multiple repositories
  - Bugs are reported in issue trackers
  - Bugs are fixed in the code
  - Code changes are recorded in the version control system
  - Code changes are reviewed with a review tool
  - Code borrows from and references Stack Overflow
  - Stack Overflow references examples from code repositories
  - Issues may be related to issues from other projects
  - …
- Studying such aspects requires combining data from different repositories

→ Establishing links between repositories a standard MSR task

Explicit links through metadata

- Systems often support explicit links
  - Separate fields that contain the linked information

- Examples:
  - Duplicates or otherwise related issues in issue trackers
  - Reference to a commit in a code review system

- Retrieval of such links can be done directly when collecting the raw data
- Still requires creating links in collected data
  - Example:
    - Issue tracker and version control system are collected separately
    - Entries for both repositories are written to a database, where they receive a unique ID from the database (e.g., Object ID, Row ID)
    - Links between the systems should go the separately ID in the database to facilitates joins

    → Requires identification of the explicitly linked content to retrieve the ID within the database

Textual links to identifiers

- Textual links to identifiers
  - An identifier is written as part of a message or comment
  - Examples:
    - Commit message: “Addresses JIRA-1234 to solve the remaining issues”
    - Comment in issue: “This was resolved in commit e34ad”
    - Comment to pull request: “This pull requests addresses the problems discussed in issue #1234”
- Textual links can be identified within text through heuristics

Heuristics to identify textual links

- Fully automated heuristic
  - Define a regular expression to look for potential links
  - Check if matched contents are actually a link (i.e., can an issue/commit/pull request/… with that ID be found)
- Examples for regular expressions for JIRA issues:
  - (?P<ID>[A-Z][A-Z0-9_]+-[0-9]+)

    Any project name Issue number
  - Variant that only works for a specific project: SPARK-[0-9]+
    - Not very robust against misspelling, should at least consider not requiring all caps
- Precision depends a lot on pattern
  - For example, Bugzilla issues are only numbers. Matching any number is not very reliable
- Recall depends a lot on robustness of pattern and message quality
  - For example, typos in project names reduce recall
- Can be tool supported
  - For example, GitHub offers auto-complete when typing # for linking of issues and pull requests

More patterns

- Bugzilla issues
  - (bug|issue|bugzilla)[s]{0,1}[#\s]*(?P<ID>[0-9]+)
- GitHub issues
  - (bug|issue|close|fixes)[s]{0,1}[#\s]*(?P<ID>[0-9]+
- Numerical identifiers
  - (\d+)
  - Often combined with other aspects, e.g., locating keywords that indicate work on a bug/issue
    - Example: (\s|^)fix(e[ds])?|(\s|^)bugs?|(\s|^)defects?|(\s|^)patch|(\s|^)issue[s]{0,1}

Texts containing hyperlinks

- Variant of textual links to identifier

- Instead of linking to an identifier and requiring knowledge about which system this may reference, a fully qualified hyperlink can be added
  - Example: “User somebody opened a pull request for this issue: https://github.com/someorga/someproject/pull/1234”

- Requires parsing the link structure to establish links
  - Similar to normal textual links, just with different patterns

Implicit links

- References are not always made explicit through identifiers or similar

- Relationships can still sometimes be inferred
  - Temporal information: if activities happened at a similar time, they might be related
  - People involved: if activities were conducted by the same people, they might be related
  - Similar texts: if activities are described using similar texts, they might be related

- Example: ReLink
  - Tries to recover links between bug fixing commits and the issues in which bugs are described
  - Requires that all three of the properties above are fulfilled
    - Must be able to map who commented on a bug to the bug fixer
      - Mapping between identities (e.g., accounts, email addresses) must be provided
    - Must have any comment within the discussion of the bug close to the time of a bugfix
      - Determined by a threshold
    - Must have a text similarity above a certain threshold
      - Based on a text processing pipeline with stop word removal, stemming, and lemmatization using the cosine similarity in a TFIDF vector space

Rongxin Wu, Hongyu Zhang, Sunghun Kim, and Shing-Chi Cheung. 2011. ReLink: recovering links between bugs and changes. In Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering (ESEC/FSE '11). Association for Computing Machinery, New York, NY, USA, 15–25

Risks of missing/additional links

- Detection of textual and implicit links fault prone
- Risks depend on purpose
  - Studies that rely on accurate links should go beyond heuristics and use manual validation
  - Examples:
    - Studying bug fixes requires accurate links between issue tracker and code
    - Relationships between issues requires accurate links within the issue tracker
    - Impact of code review requires accurate links between code review, code, and often also issues to consider subsequent code quality
- Regardless of the purpose, noise should be estimated
  - How much is there in addition?
  - How much is overlooked?
  - Often also depends on algorithm:
    - Is the heuristic restrictive and designed to prevent false positives?
    - Is the heuristic inclusive and designed to prevent false negatives?
    - How is this trade-off balanced?
- Manual validation can help to determine and mitigate risks
  - Determine: manually label a sample, calculate noise
  - Mitigate: manually validate data to reduce noise

Collecting defect data

MSR and Quality

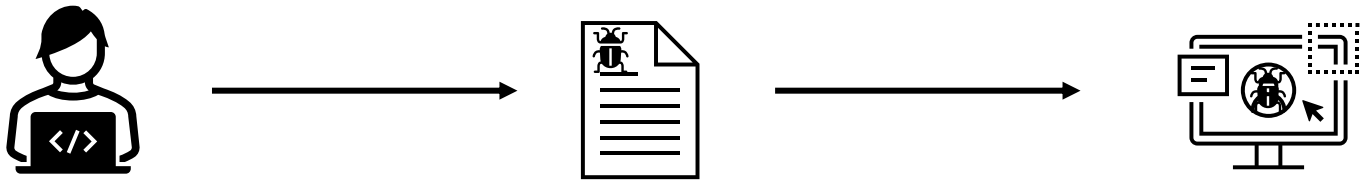
- MSR often interested in quality
  - Understanding quality
  - Measuring quality
  - Improving quality
  - Predicting quality

- Internal vs external quality
  - Internal quality considers the quality of source code
    - Code smells
    - Architecture
    - Testability
    - …
  - External quality considers the quality of the resulting software
    - Required features available
    - Number of bugs
    - Robustness
    - …

Defect, Bug, Failures, Fault, Error

- Error
  - human action that produces an incorrect result
- Fault, bug
  - manifestation of an error in software
- Failure
  - termination of the ability of a system to perform a required function or its inability to perform within previously specified limits
  - an externally visible deviation from the system's specification
- Defect
  - imperfection or deficiency in a work product where that work product does not meet its requirements or specifications and needs to be either repaired or replaced

  → Usually used as synonym for fault and bug



Developer makes an error | Code contains fault | Failure during the execution

Quality often measured through defects

- MSR has difficulties with analysis of errors
  - Sometimes possible, if errors are discussed, e.g., within issues or code review
- Detection of failures the task of software testing
  - Requires execution of the software and not within the domain of MSR
- Reporting of failures can be analyzed by MSR
  - Data contained within the issue tracker
- MSR well-suited to analyze bugs
  - Reported indirectly through failures within the issue tracker
  - Manifest in source code
  - Changes that introduce and fix bugs are tracked within the VCS

Information about bugs

- Reported bugs
  - Analyze data from the issue tracker
  - Most issue trackers have a specific field to identify the type of issue
    - Allows explicit labeling of issues as bug
    - Some tools do not have this (e.g., GitHub, GitLab) and use labels instead
- Bug fixes
  - Commits within the VCS that remove the bug from the code
- Bug introducing changes
  - Commits within the VCS that introduce a new bug into the code
- Buggy code
  - A snapshot of the software that contains a known bug, including knowledge about the location of the bug
  - Often paired with a clean version that does not contain the bug

Labeling with keywords

- Use keywords to identify bug fixing commits
  - Use regular expressions to scan commit messages for specific key words
  - Examples for keywords
    - bug, fix, error, fault, failure, issue, patch
- Keywords lack context
  - E.g., “We added some tests to prevent future bugs” would be wrongly identified as bug fixing
  - Leads to false positives, i.e., too many commits are identified as bug fixing
  - Overestimate
- Keywords may miss commits not perfect
  - E.g., “Aligns code with prior changes to work correctly” would not be captured by the above keywords
  - Leads to false negatives, i.e., missing bug fixing commits
- More keywords (e.g., correct) reduce false negatives, but increase false positives
  - Typical recall/precision trade-off



https://github.com/apache/spark/commit/5f999b0e06da8457a39678b9063ae4a47ae10f71

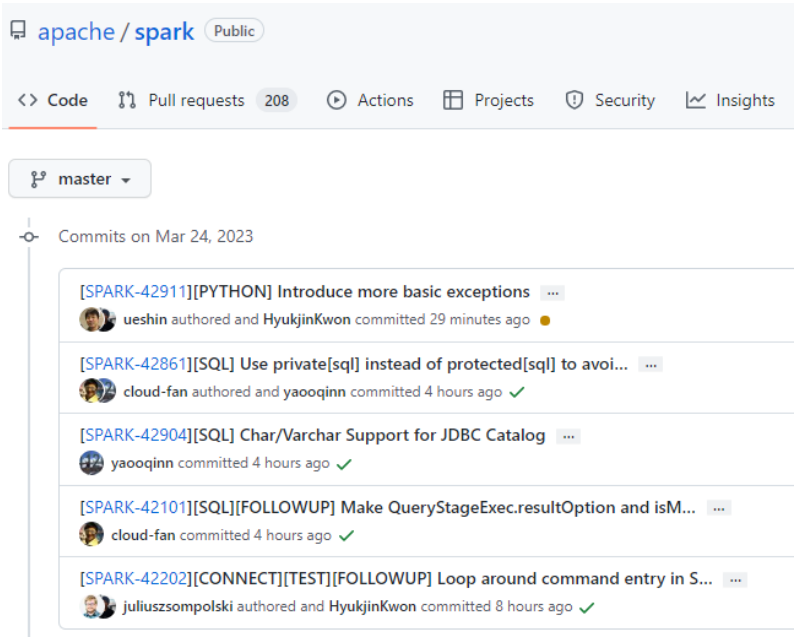
Labeling based on links

- Exploit links to issue tracker to identify bug fixes
  - If a commit references a bug issue, the commit is likely a bug fix
- Avoids issues of keyword matching
  - Does not depend on single words without context anymore
- Makes assumptions on the development process
  - Active use of issue tracker to track all bugs
    - Any non-tracked bug is missed
  - Existence of links between commits and bugs
    - Any non-linked issue/commit misses bugs
- Violating these assumptions leads to false negatives, i.e., missing bugs

Requirements for link-based labeling

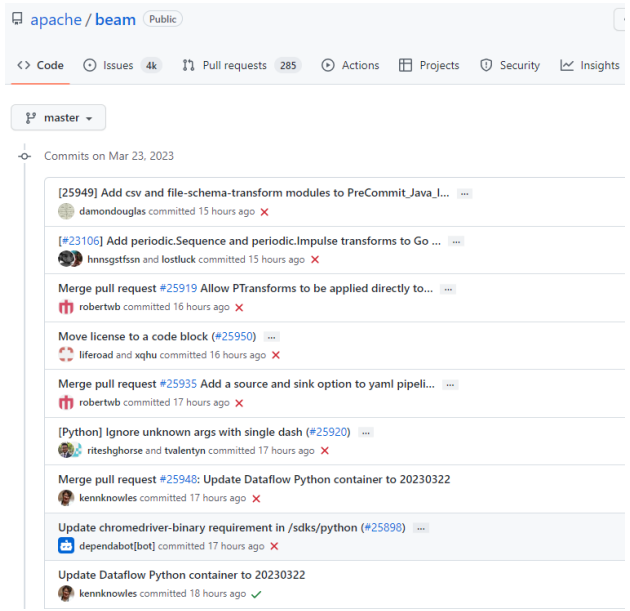
- Quality of data depends on rigor of developers linking issues
  - If issues are not created, typed as bug, or not linked, this approach fails
- Depends on project!

https://github.com/apache/spark/commits/master



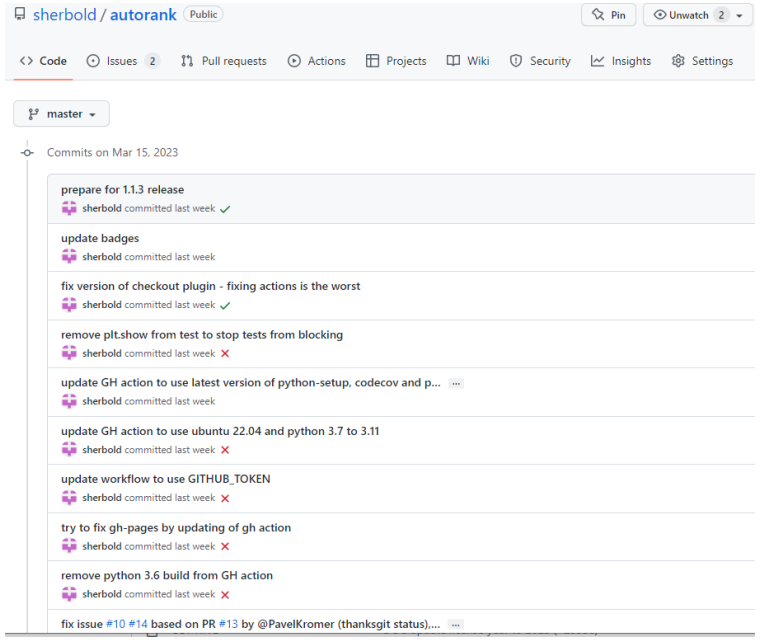
Sometimes very accurate

https://github.com/apache/beam/commits/master



Sometimes mixed

https://github.com/sherbold/autorank/commits/master



Sometimes sporadic

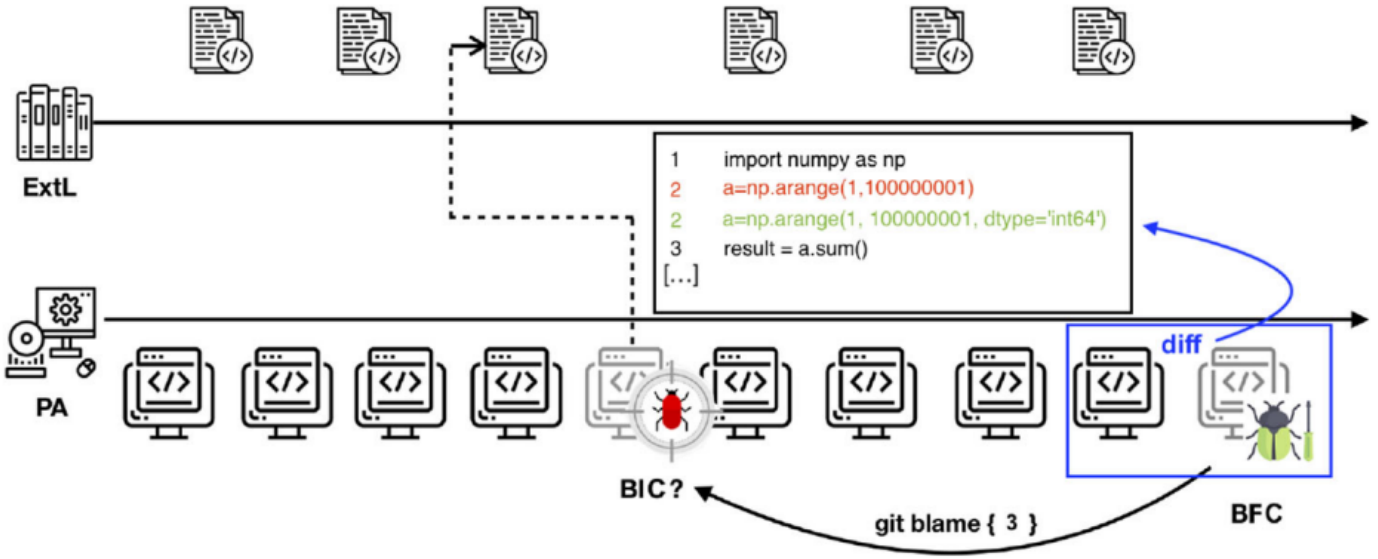
Identifying bug introducing changes

- Generic approach
  - Find bug fixing commit
  - Consider what was changed within the bug fix
  - Use version control system to identify when this was last changed

- The above generic approach has many assumptions about bugs
  - The last change to every line in the bug fix was equally responsible for introducing the bug
  - No change to any other line was responsible for introducing the bug
  - There were no changes to these lines between the bug introduction and the bug fix

- Assumptions obviously do not always hold
  - Any such heuristic needs to deal with noise!

Visualization of the bug introduction



1. Create diff for bug fixing commit

2. Use git blame to identify bug introduction

Rodríguez-Pérez, G., Robles, G., Serebrenik, A. *et al.* How bugs are born: a model to identify how bugs are introduced in software components. *Empir Software Eng* **25**, 1294–1340 (2020). https://doi.org/10.1007/s10664-019-09781-y

The SZZ algorithm

- Named after the authors Śliverski, Zimmermann, and Zeller
- Combines issue linking with finding bug fixes and bug introducing changes

- Consists of two phases
  - In the first phase, bug fixes are identified
    - Based on a combination of number-based issue linking (syntactic check) and heuristics to validate the link like keywords or author matching
  - In the second phase, bug introducing changes are identified
    - Based on finding last changes to lines and plausibility filters to discard lines that are deemed unlikely as introducing for the bug
- Has been around for almost as long as MSR exists and is used within many papers

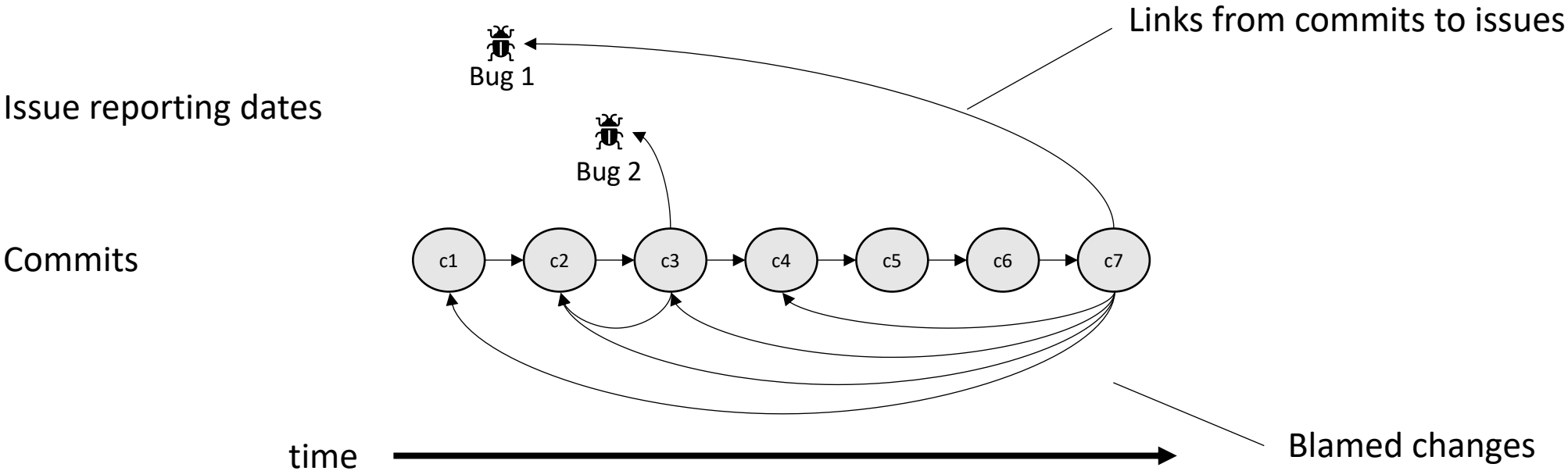
Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? In Proceedings of the 2005 international workshop on Mining software repositories (MSR '05). Association for Computing Machinery, New York, NY, USA, 1–5. https://doi.org/10.1145/1083142.1083147

The two phases of SZZ

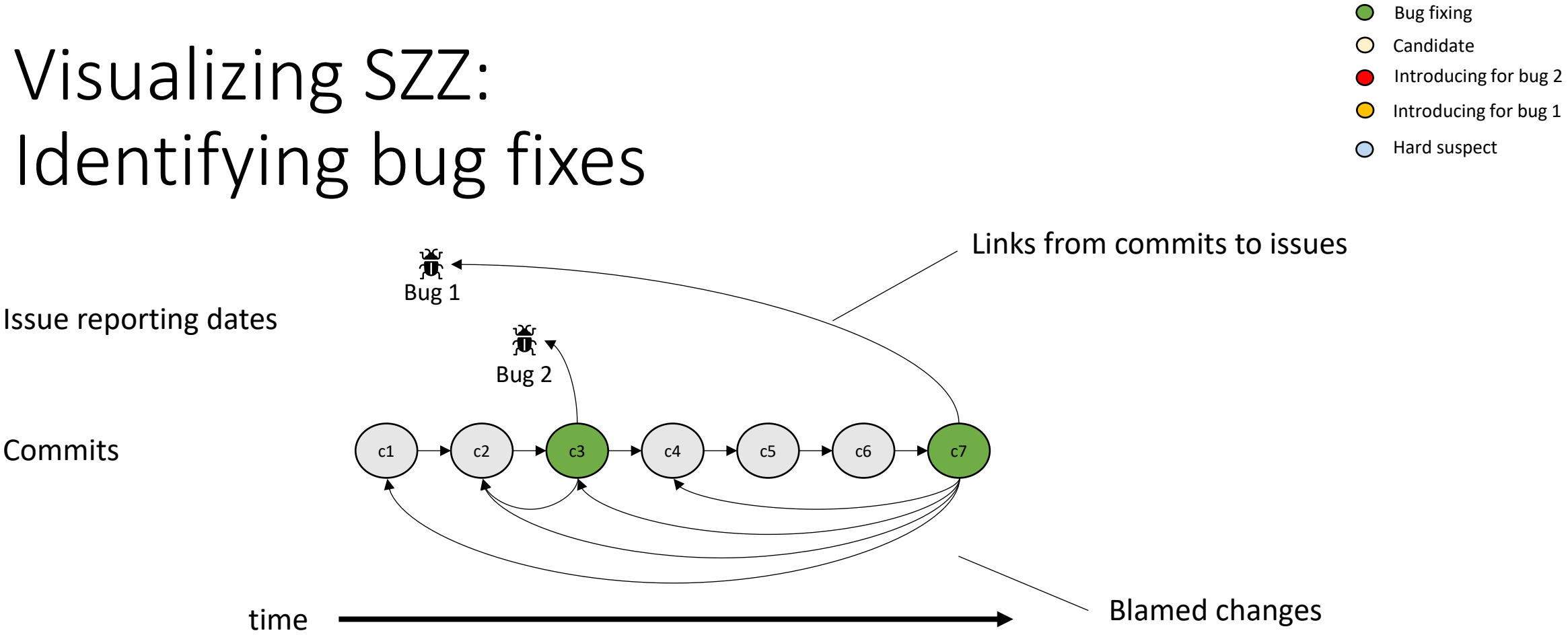
1. Determine if there is any number within the commit. If a number is found and there is a bug issue matching this number, the commit is identified as bug fixing, if
   - at least two of the following properties are fulfilled:
     - the issue was resolved as fixed at least once
     - the author of the commit is assigned to the issue
     - the title or description of the issue is contained in the commit message
     - one or more files that are changed by the commit are attached to the issue
   - only one of the above properties is fulfilled and additionally, one of the following is fulfilled
     - the commit message contains a term like bug or fix
     - the number in the commit is clearly identifiable as linking to an issue, e.g., because it starts with “Bug #NUMBER”
     - the commit only contains numbers
2. For each line that was changed in the fixing commit, identify the last change to that line as *candidate* for an introducing change. Candidates are considered to be bug introducing, if
   - the date of the change was before the reporting date of the issue
   - all other candidates are *suspect*
     - suspects that are part of a bug fixing commit are considered as bug introducing due to the *partial fix* rule of SZZ
     - suspects that are part of a commit that has other bug introducing changes are considered as bug introducing due to the *weak suspect* rule of SZZ
     - all other suspects are not considered to be bug introducing and called *hard suspects*

Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? In Proceedings of the 2005 international workshop on Mining software repositories (MSR '05). Association for Computing Machinery, New York, NY, USA, 1–5. https://doi.org/10.1145/1083142.1083147

Visualizing SZZ: Sample data

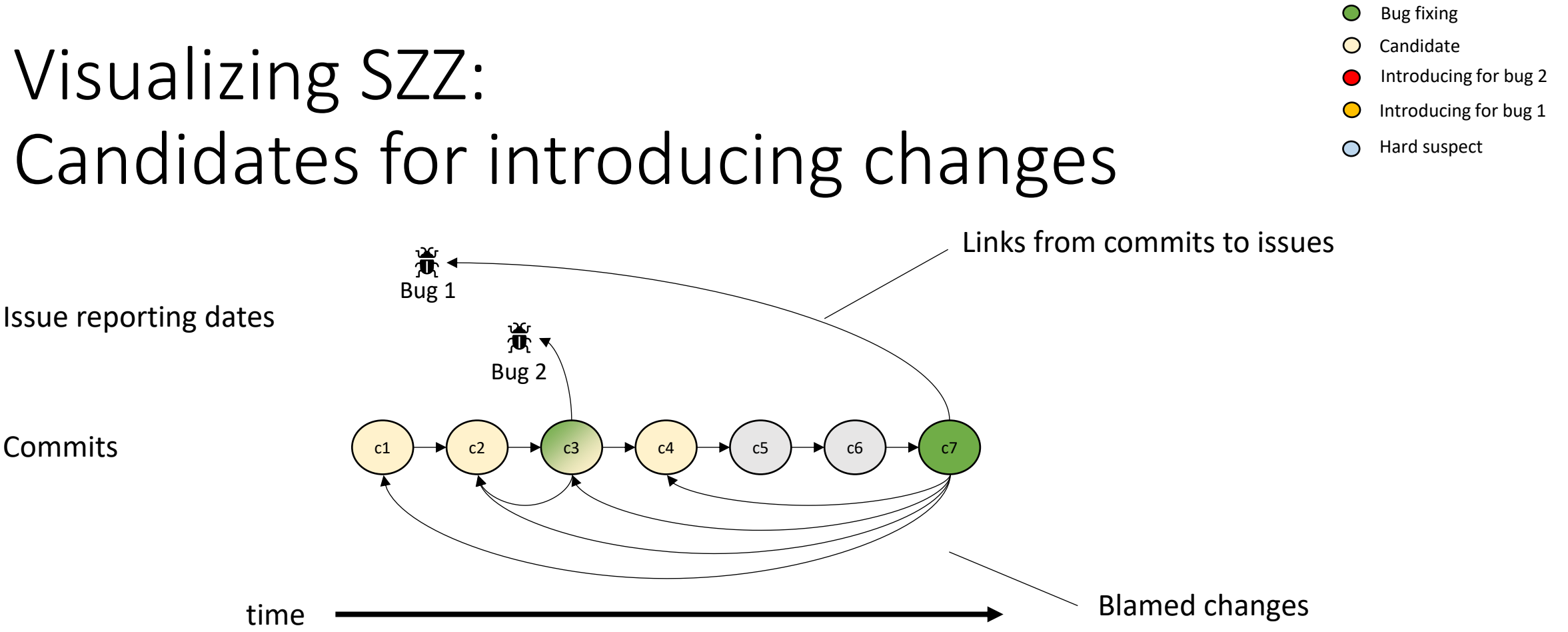


Visualizing SZZ: Identifying bug fixes



- c7 is fixing bug 1
- c3 is fixing bug 2

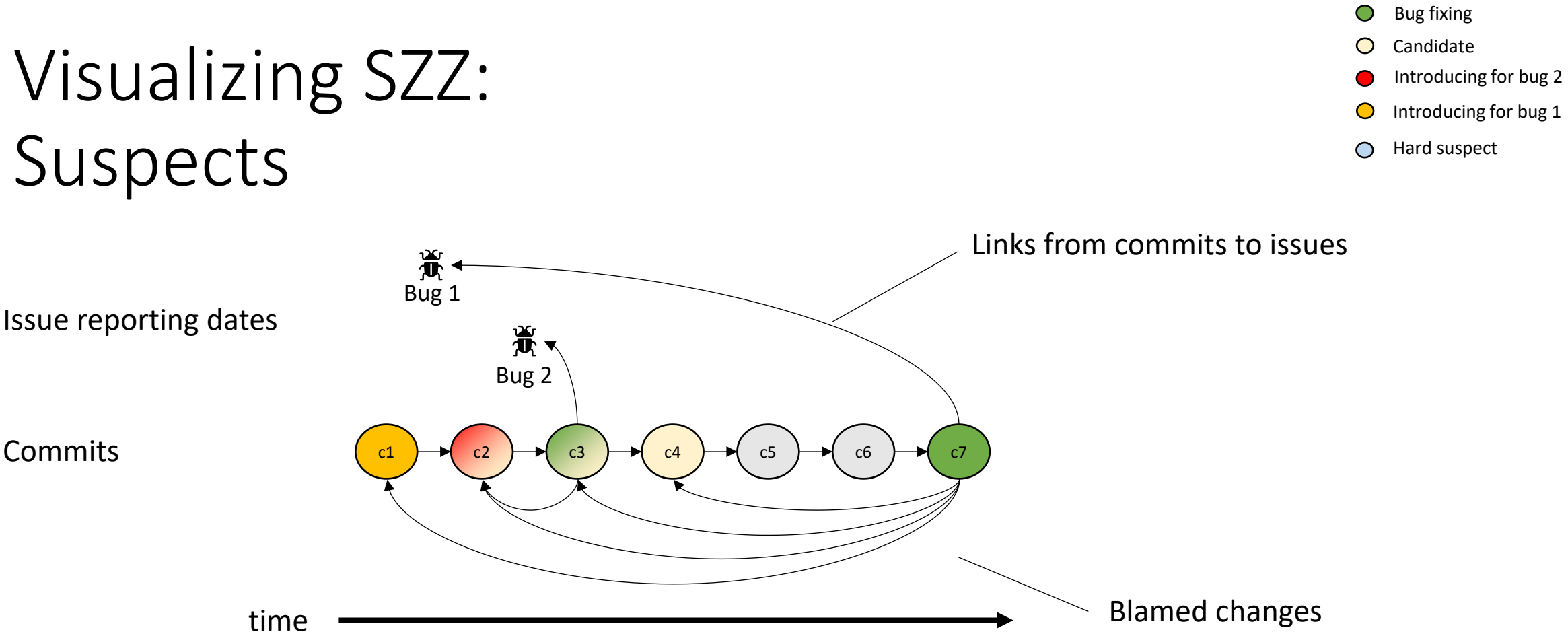
Visualizing SZZ: Candidates for introducing changes



- c7 is fixing bug 1
- c3 is fixing bug 2

- c1, c2, c3, and c4 are candidates for bug 1
- C2 is candidate for bug 2

Visualizing SZZ: Suspects

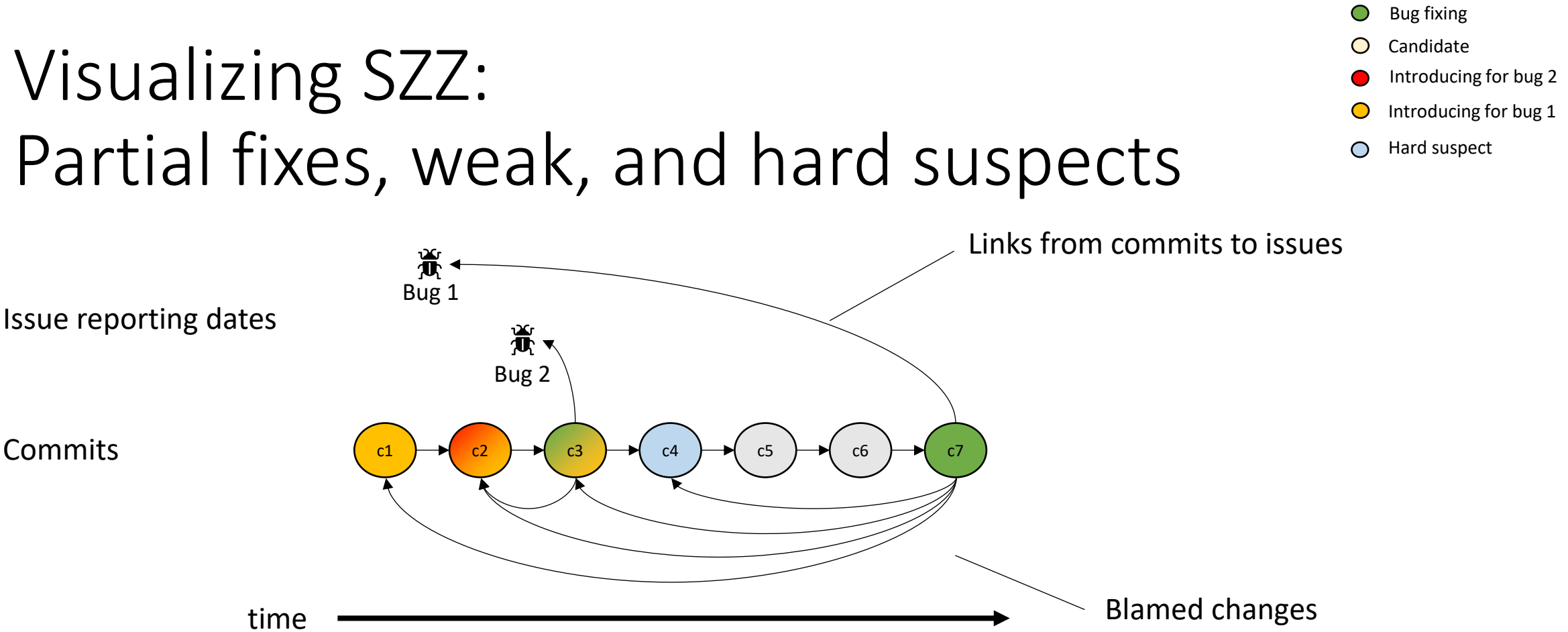


- c7 is fixing bug 1
- c3 is fixing bug 2

- c1, c2, c3, and c4 are candidates for bug 1
- C2 is candidate for bug 2

- c1 is before the reporting date of bug 1 and bug introducing for bug 1
- c2, c3 and c4 are after the reporting date and suspects for bug 1
- c2 before the reporting date of bug 2 and bug introducing for bug 2

Visualizing SZZ: Partial fixes, weak, and hard suspects



- c7 is fixing bug 1
- c3 is fixing bug 2

- c1, c2, c3, and c4 are candidates for bug 1
- C2 is candidate for bug 2

- c1 is before the reporting date of bug 1 and bug introducing for bug 1
- c2, c3 and c4 are after the reporting date and suspects for bug 1
- c2 before the reporting date of bug 2 and bug introducing for bug 2
- c2 is a weak suspect for bug 1, because it is introducing for bug 2
- c3 is a partial fix for bug 1
- c4 is a hard suspect for bug 1, because it is not introducing or fixing for another bug

Improvements of SZZ

- Many improvements suggested to the SZZ algorithm
  - Exclude changes to whitespaces and comments
    - Cannot cause bugs (in most languages)
  - Use refactoring detection algorithm (e.g., RefactoringMiner or RefDiff) to exclude refactorings
    - Refactorings, by definition, do not modify the logic and, therefore, cannot cause bugs
  - Ignoring meta-changes, like copying the complete file path or modifying file permissions

- Basic SZZ variant still in use
  - Study from 2018 shows that SZZ without changes is still used by 40% of papers using SZZ

Da Costa DA, McIntosh S, Shang W, Kulesza U, Coelho R, Hassan AE (2017) A framework for evaluating the results of the szz approach for identifying bug-introducing changes. IEEE Transactions on Software Engineering 43(7):641–657.

Kim S, Zimmermann T, Pan K, Jr Whitehead EJ (2006) Automatic identification of bug-introducing changes. In: 21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06), pp 81–90

Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig, "RefactoringMiner 2.0," *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 930-950, March 2022.

Silva D, Valente MT (2017) Refdiff: Detecting refactorings in version histories. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), pp 269–279

Rodríguez-Pérez G, Robles G, González-Barahona JM (2018) Reproducibility and credibility in empirical software engineering: A case study based on a systematic literature review of the use of the szz algorithm. Information and Software Technology 99:164–176.

Problems of SZZ

- As a heuristic, SZZ is full of assumptions
  - Numbers in commit messages are usually related to issues
  - Developers reference bugs within the commit message
  - Developers label issues correctly
    - Bugs are bugs, features are features, …
  - Every line that was touched in a bug fix is related to the bug
  - Every last change to a line that was touched to fix a bug may have been responsible for its introduction
  - Bugs were introduced by the last change to a line and not already dormant for a longer period of time
  - Changes after the bug reporting data can still be responsible for the bug
- These assumptions do not always hold!
  - Some overlap with what we discussed already for linking of commits to issues
- Each is a source of noise

Herbold, S., Trautsch, A., Trautsch, F. et al. Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection. Empir Software Eng 27, 42 (2022).

Noise in heuristic MSR data

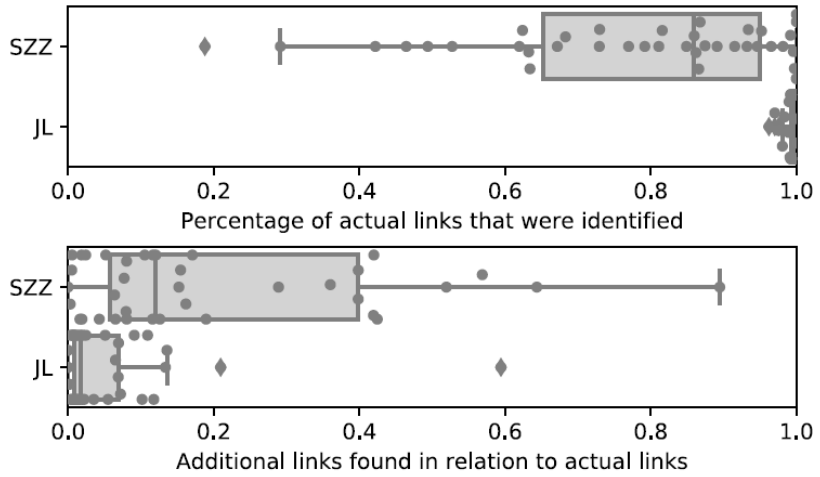
Validity of MSR depends on noise

- As discussed, wrong assumptions on what the data should be, lead to noise when heuristics are used
- Amount of noise impacts validity
  - Noisy data can still be analyzed
  - It is possible to draw valid conclusions or build tools based on noisy data
  - Nevertheless: analysis needs to take noise into account (!)
- We need to understand noise to understand how to take it into account and how it affects results
  - Pick data sources that have likely less noise, given the properties of the noise
  - Use analysis approaches capable of mitigating the noise
  - Evaluate results with respect to the known threats to validity based on the noise
- Example on the next slides:
  - Noise in SZZ

Noise in commit-issue links

SZZ: match numbers within commit messages to issues (e.g., 1 to NUTCH-1)

JL: match complete Jira identifier to issues (e.g., NUTCH-1)



Data is based on 18,721 manually validated links

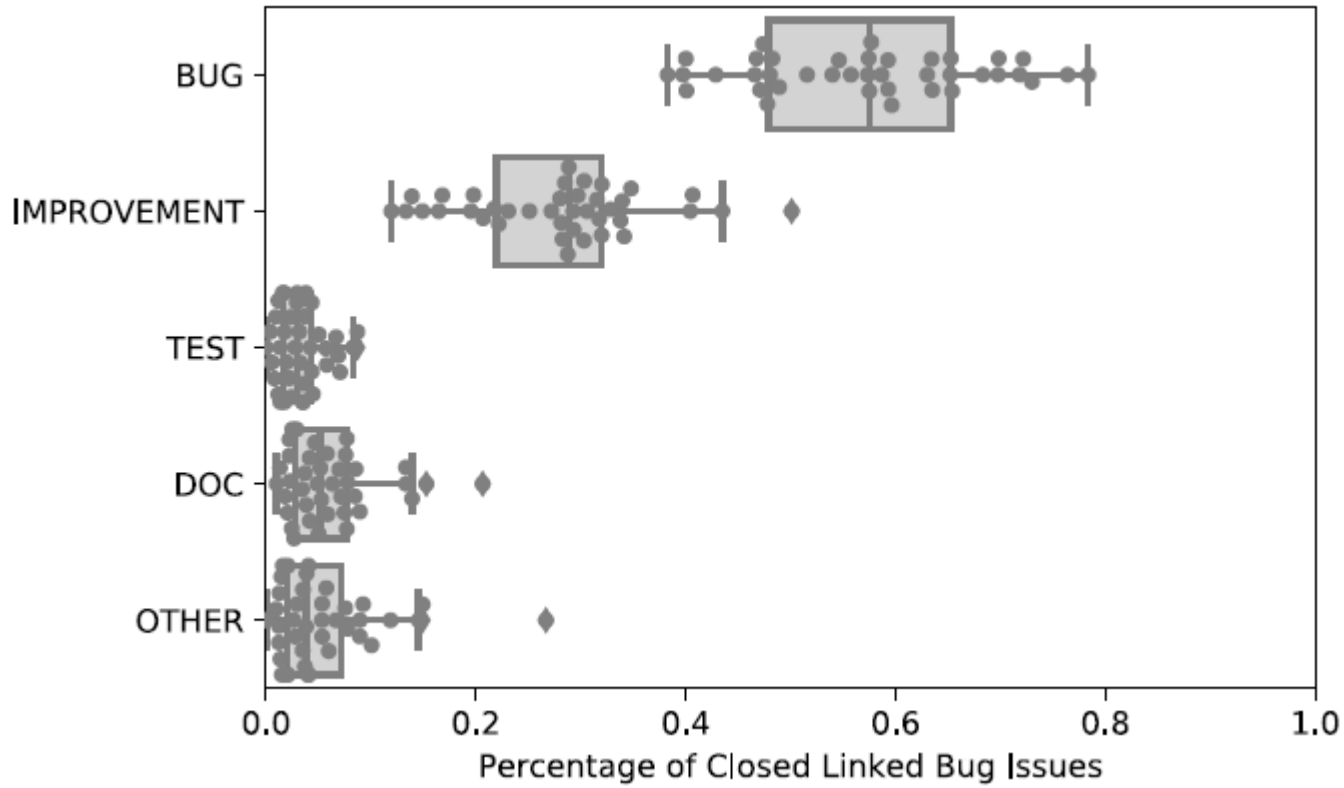
Herbold, S., Trautsch, A., Trautsch, F. et al. Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection. Empir Software Eng 27, 42 (2022)

Noise in issue types

Category	Description
BUG	Issue reports documenting corrective maintenance tasks that require semantic changes to source code.
RFE	Issue reports documenting an adaptive maintenance task whose resolving patch(es) implemented new functionality (request for enhancement; feature request).
IMPR	Issue reports documenting a perfective maintenance task whose resolution improved the overall handling or performance of existing functionality.

Plot on the right uses RFE and IMPR together as IMPROVEMENT

Median of 42% mislabeled as bug!



Data is based on 11,295 issues that are linked to a commit

K. Herzig, S. Just and A. Zeller, "It's not a bug, it's a feature: How misclassification impacts bug prediction," *2013 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 392-401

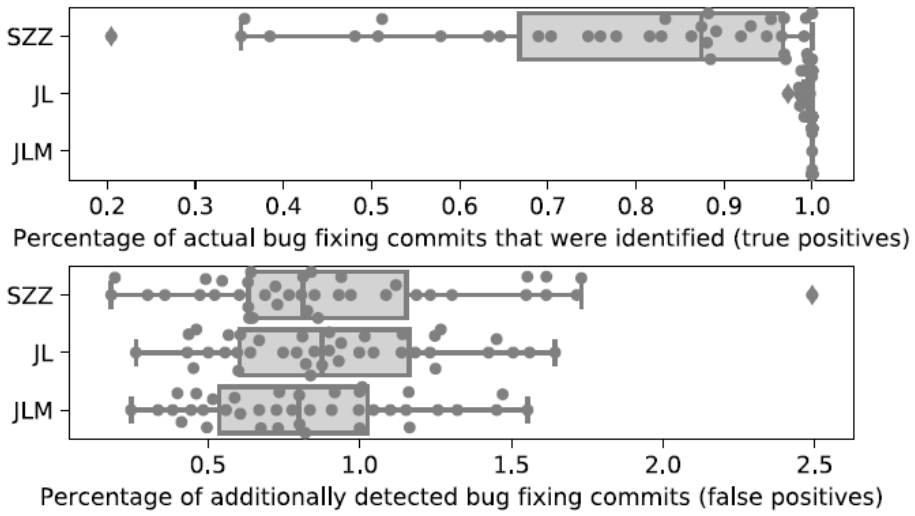
Herbold, S., Trautsch, A., Trautsch, F. et al. Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection. Empir Software Eng 27, 42 (2022)

Resulting noise in bug fixing commits

SZZ: match numbers within commit messages to issues (e.g, 1 to NUTCH-1)

JL: match complete Jira identifier to issues (e.g., NUTCH-1)

JLM: manually validated link from commit to issue



Data is based on the manually validated links and issue types

SZZ misses about one in five bug fixing commits and only about half of the commits identified as bug fixing are actually bug fixing!

Better issue linking can only avoid missing bugs, the source are wrong issue types!

This noise affects bug introducing commits as well!

Herbold, S., Trautsch, A., Trautsch, F. et al. Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection. Empir Software Eng 27, 42 (2022)

Tangled changes in bug fixes

General information

Revision	Show 26c458f09bc858537d02b3749868e8318b167a68 on GitHub
Issue Links	IVY-1150
Commit Message	FIX: Artifact report throws NPE when artifact is not in cache (IVY-1150) (thanks to Steve Jones) git-svn-id: https://svn.apache.org/repos/asf/ant/ivy/core/trunk@892370 13f79535-47bb-0310-9956-ffa450edef68

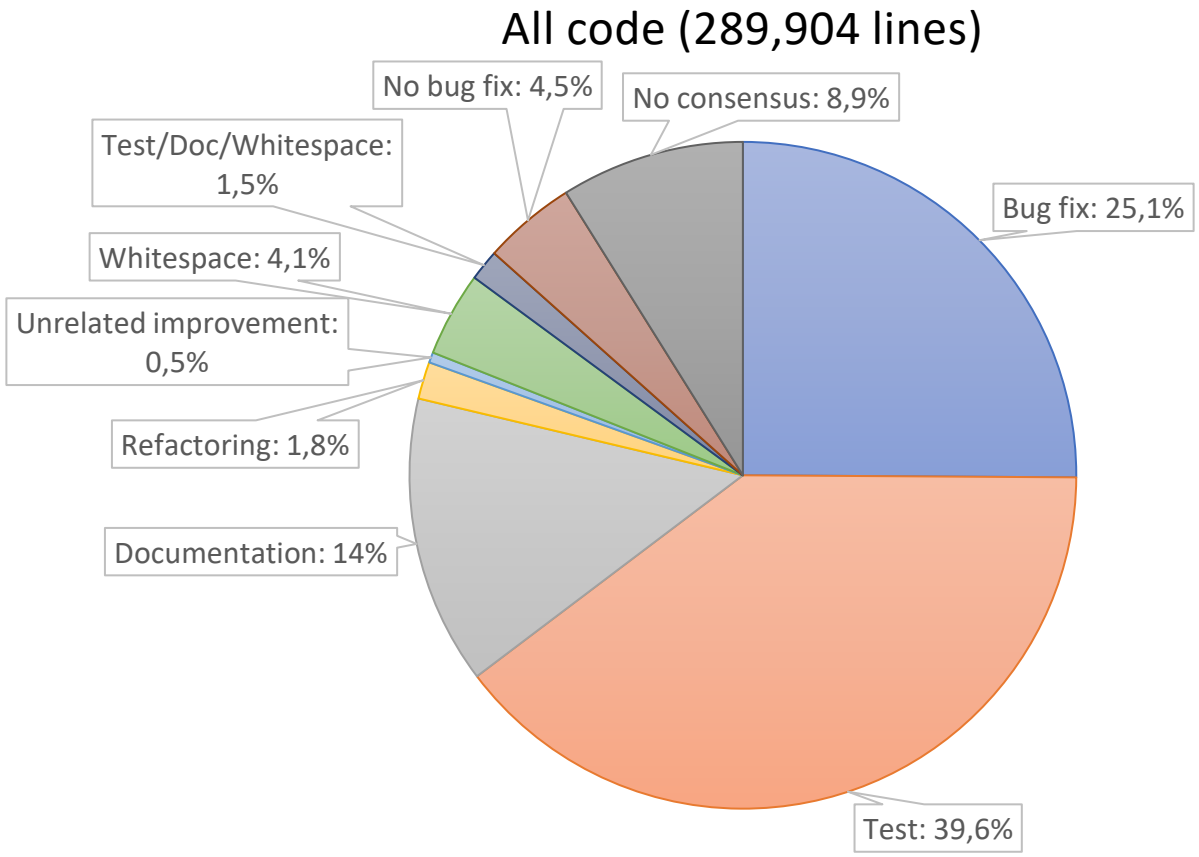
Changes

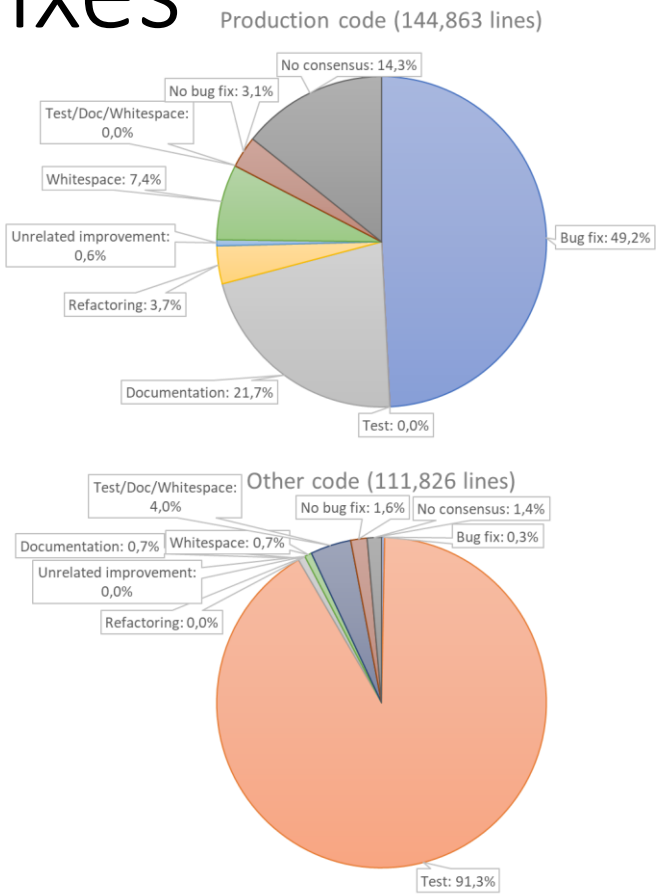
```
1  /*…                                          1  /*…
155                                             155
156                writeOriginLocationIfPresent(cac  156                writeOriginLocationIfPresent(cac
                                                157 +                    writeCacheLocationIfPresent(cach
157                                             158
158 -              writeCacheLocation(cache, saxHan
159 -
160                Set artifactDestPaths = (Set) ar  159                Set artifactDestPaths = (Set) ar
161                for (Iterator iterator = artifac  160                for (Iterator iterator = artifac
244        }                                        161                    .hasNext();) {…
245                                                 244
246 -      private void writeCacheLocation(RepositoryCacheManager c  245 +      private void writeCacheLocationIfPresent(RepositoryCache
247              ArtifactDownloadReport artifact) throws SAXExcep  246              ArtifactDownloadReport artifact) throws SAXExcep
248          File archiveInCache = artifact.getLocalFile();  247          File archiveInCache = artifact.getLocalFile();
249                                                 248
                                                249 +        if (archiveInCache != null) {
```

Which lines contribute to the fix?



Amount of tangling in bug fixes





Assuming changes address single aspects is very often wrong!

Herbold, S., Trautsch, A., Ledel, B. et al. A fine-grained data set and analysis of tangling in bug fixing commits. Empir Software Eng 27, 125 (2022). https://doi.org/10.1007/s10664-021-10083-5

Noise in detection of bug introducing changes

Table 6 Ground truth data for inducing changes and release assignments

Fixed Issue	Inducing Issue	Time to Fix	Deviation JLMIV+R	Affected Versions Field	Actually Affected Releases	6M	AV	IND
IVY-882	IVY-857	45 days	0 days	2.0-RC1	-	✓	(✓)	✓
CALCITE-2253	CALCITE-2206	5 days	0 days	-	-		✓	✓
CALCITE-1215	CALCITE-1212	0 days	0 days	-	-		✓	✓
CALCITE-822	CALCITE-783	17 days	0 days	-	-		✓	✓
KYLIN-3223	KYLIN-3239	1 days	0 days	2.2.0	-			✓
NUTCH-683	NUTCH-676	20 days	0 days	1.0.0	-	✓	(✓)	✓
PARQUET-214	PARQUET-139	54 days	-29 days	1.6.0	-	✓	(✓)	✓
TIKA-599	TIKA-528	147 days	0 days	0.9	0.8, 0.9	✓		✓
TIKA-2483	TIKA-2311	196 days	0 days	1.16	1.15, 1.16	✓		✓
SYSTEMML-1126	SYSTEMML-584	323 days	-7 days	-	0.10, 0.11, 0.12, 0.13			✓
SYSTEMML-2162	SYSTEMML-1919	163 days	0 days	-	1.0.0			✓
SYSTEMML-2275	SYSTEMML-2217	323 days	0 days	-	-		✓	✓
LENS-538	LENS-486	0 days	0 days	2.2	-	✓	(✓)	✓
KNOX-1134	KNOX-1119	2 days	0 days	-	-			✓
VALIDATOR-376	VALIDATOR-273	474 days	-174 days	1.4.1	(1.4.1)	✓	✓	✓
GIRAPH-918	GIRAPH-908	2 days	0 days	-	-	✓		✓
GIRAPH-832	GIRAPH-792	1 days	0 days	-	-	✓		✓
GIRAPH-88	GIRAPH-11	0 days	0 days	0.1.0	-	✓	(✓)	✓
GIRAPH-34	GIRAPH-27	4 days	-1 days	0.1.0, 1.0.0	-	✓	(✓)	✓
EAGLE-573	EAGLE-569	0 days	0 days	0.5.0	-	✓		✓

The affected version for VALIDATOR-376 is in braces because it is minor version, which we omitted otherwise

Sample shows that using git blame often works but can be very unreliable.

Other data from issue tracker (e.g., affected versions field) even more unreliable!

Other literature agrees that the last change is correct about 70% of the time

JLMIV+R: Date of last bug introducing change. Uses manually validated bug fixes, git blame ignores whitespaces, comments, and refactorings

Herbold, S., Trautsch, A., Trautsch, F. et al. Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection. Empir Software Eng 27, 42 (2022)
Rodríguez-Pérez, G., Robles, G., Serebrenik, A. *et al.* How bugs are born: a model to identify how bugs are introduced in software components. *Empir Software Eng* **25**, 1294–1340 (2020).

Reasons for this noise

- Simplified assumptions
  - E.g., introduction of bug is in change prior to addition of bug

- Misalignment of developer interests and MSR goals

  "The question of whether an issue is a bug or not is a hard one. And the definition of a bug not only differs between users and developers but also between developers themselves. In principle, if an issue bothers the user, the developer should fix it, whether or not he considers it to be a bug or not. From this perspective, the issue report category does not matter. But a data miner building a defect prediction model must distinguish between bugs and non-bugs. Otherwise, the prediction model would predict changes, not defects" – Herzig et al. (2013)

- Same reasoning also applies to tangling

K. Herzig, S. Just and A. Zeller, "It's not a bug, it's a feature: How misclassification impacts bug prediction," *2013 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 392-401

Dealing with noise from heuristics

- Option 1: Ignore the noise
- Option 2: Accounting for the noise during the data collection
- Option 3: Account for the noise during the analysis
- Option 4: Manually clean the data
- Option 5: Improve heuristics to reduce the noise, then proceed with one of the other options

Ignoring noise

- Easy low effort solution
- Never ideal from a validity point of view, but can still be a good solution. This depends on multiple factors, e.g.,
  - Amount of noise
    - A small amount of noise is less likely to affect results than a large amount of noise
  - Costs of removing the noise
    - If removing the noise is too expensive, ignoring may be the only option
  - Type of analysis
    - Some analysis approaches are inherently better at dealing with noise, e.g., many statistical methods
    - Noise can often (safely) be ignored when fitting models, but cannot easily be ignored when evaluating models
    - (but depends on type of noise, see below)
  - Strength of the effect that is studied because weak effects are more likely hidden by noise than strong effects
    - Can possibly still detect effects, but estimates of the strength (e.g., effect size) may be unreliable
  - Type of noise
    - If noise is randomly distributed (e.g., Gaussian noise) this is easier to ignore than noise in a specific direction, e.g., many false positives and few false negatives

    → Noise may introduce biases

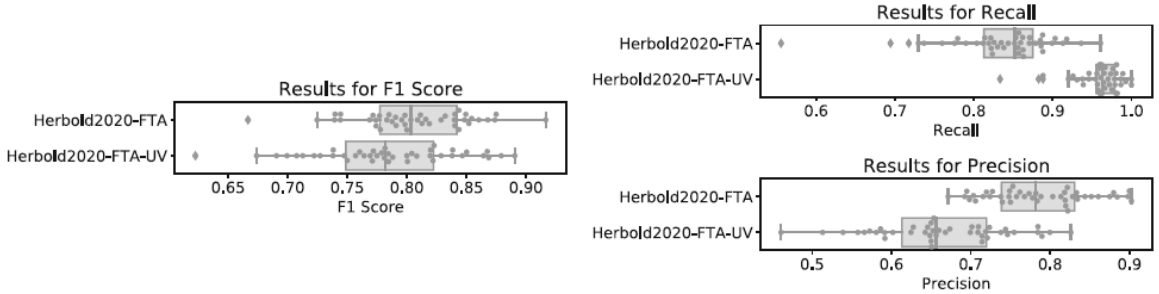
Example for safely ignoring noise

- Training machine learning models can be done in presence of noise
  - Theoretical frameworks like the statistical query model even proof that optimal solutions can still be found if certain conditions are fulfilled
  - Usually more data is required to mitigate the negative impact of noise
  - Depending on the cost and amount of noise, collecting more noisy data may be more efficient than less clean data for training

- Evaluation of machine learning models would be biased by noise
  - This would enable overfitting on noise (i.e., models could learn noise and we would not detect this=
  - This could lead to wrong performance estimations. Type of mistake depends on noise
    - Underestimation of performance because noisy data would be inadvertently counted as mistakes, even though the true label would be predicted correctly
    - Overestimation of performance if both the model and the noise are biased in a certain direction (e.g., prediction of too many positives and many false positives in the test data)
- Feasible solution:
  - Train with large amounts of possibly noisy data, evaluate with smaller amount of clean data

Training of models for the prediction of bug and non-bug issues

- Goal:
  - Automatically determine if a reported issue is a bug or not
- Problem:
  - Data in repositories is often mislabeled
  - Cleaning this data requires a large amount of effort
- Solution:
  - Train a machine learning model on a large amount of easily ava
  - Evaluate this model on a smaller amount of cleaned data



- Works, but shows differences:
  - FTA is a fastText model trained on a smaller cleaned data set. This leads to higher precision, but at the cost of recall because less data was available.
  - FTA-UV was trained on a large amount of noisy data instead. The noise reduced precision, but the larger data set helped to improve the recall

Herbold, S., Trautsch, A. & Trautsch, F. On the feasibility of automated prediction of bug and non-bug issues. *Empir Software Eng* **25**, 5333–5369 (2020). https://doi.org/10.1007/s10664-020-09885-w

Accounting for noise during the data collection

- When we know the noise and its sources, we can sometimes avoid it

- Example:
  - When we require issue links to determine bugs, commits that do not link to issues cannot be bug fixing
  - We can determine how many commits in a project link to issues → high ratio of linking commits means that this development practice was followed → Likely leads to a higher data validity
  - We can measure this for projects and only analyze projects with high linking ratios

- Good strategy to mitigate noise by ensuring that the assumptions on the development process hold

- Introduces biases into the data
  - In the above example, we would pick only projects that have strict guidelines regarding commit messages and issue reporting
  - Such projects often have other quality assurance measures in place and, in general, a high-quality process
  - This affects results and it is unclear how results transfer to projects that do not have such a rigorous process

Accounting for noise during the analysis

- Analysis of data can try to account for or avoid noise

- White noise is for many statistical approaches not a problem and already accounted for
  - Noise is equal in all directions and randomly distributed
  - Reduces power of statistics and can be dealt with through more data
  - Usually modelled as a random variable with an expected value of zero
    - Example: addition of a random variable of with expected value of zero does not affect calculations of the mean

- Filtering of outliers helps to remove implausible data points
  - For example, data from 1970 usually indicates a broken timestamp, because timestamps are often measured in second since January 1st, 1970 (Unix Time)
  - Such noise is easy to detect and would have a strong influence on analysis

- Considering noise when discussing results is also important
  - But without further action, this is similar to ignoring noise – because the impact of potential noise should still be discussed

Manually cleaning the data

- Often, humans can identify the noise
  - We can read issues and related information to understand the type of the issue
  - We can analyze code to understand changes
  - We can inspect suspicious data points (e.g., extreme values) and determine if they are plausible or why they might not be trustworthy
- Usually most accurate solution to deal with noise
- Very expensive and labor intensive
  - Humans also make mistakes. Usually requires multiple people to clean to data, to achieve best results. Otherwise, some bias may be left.
  - Humans need training. The above tasks require knowledge about software engineering and programming. People with such training often have academic degrees and employing them for such tasks is expensive.
  - Human effort does not scale well. Both recruiting humans as well as the costs involved limits what is possible.
- Need to determine the trade-offs and if this effort is worth the benefits

…more on this later, when we talk about coding of data

Improve heuristics

- The best way to deal with noise from heuristics is just better heuristics
  - Reduces the general problem
  - Can be used by others as well
- These heuristics are an active research topic on their own!
  - E.g., lots of papers on SZZ
  - Any improvement must be validated to measure the effect. Ideally, the impact on prior research should also be determined (i.e., do results change, when a better heuristic and cleaner data is used?)
- Different approaches to improve heuristics
  - More complex processing of data, e.g., we could use the machine learning model for the prediction of bug and non-bug issues to reduce the noise in data about bug fixing commits in a link-based approach
    - Common approach that sometimes has low-hanging fruits. E.g., the original SZZ did not ignore whitespaces. Ignoring whitespaces was an easy improvement through more complex data processing
  - Removing assumptions on the development process to make approach more generic, e.g., we could ignore issue types since we know they are a source of noise and try to infer if a change was bug fixing from other data, e.g., the commit message, the code change, or even dynamic data like test execution data
  - Include automated data cleaning steps like outlier detection in the heuristic
  - Include measures for the certainty of the heuristic in the correctness of the data (similar to scores for machine learning algorithms
    - Enables subsequent manual analysis and correction of only the data with a low certainty

Coding of data

Coding of qualitative data

- Qualitative analysis considers the interpretation of data (e.g., text, images) by humans
- Coding of data means to assigned *codewords* (=labels) to data
  - We already did this! Validation if an issue is a bug is a type of coding
- The codewords are maintained in a *codebook*
- Humans assigning the codewords to data are called *raters*
- Deductive coding and inductive coding
  - Deductive coding works with a fixed and pre-defined codebook
  - Inductive coding works with a dynamic code book that can be extended by the raters

Dealing with human biases and noise

- Humans are imperfect
- We make random mistakes
  - E.g., accidentally clicking a wrong button
- We make actual mistakes
  - Due to lack of skill or missing important information
- Reasoning and interpretation may be different between individuals
  - Does not automatically mean that one may be right, could also just be differences of opinion
- Needs to be accounted for when humans code data!

Multiple raters

- Using multiple raters to conduct the same tasks helps to prevent noise and deal with biases
- Many strategies possible, depend on data, task, and required accuracy of data
- Some common strategies are:
  - Multiple raters assign labels for the same task and assign the majority label
    - Works well if there are few labels (ideally binary)
    - Many labels would require many raters per label to ensure finding a reasonable majority
  - Multiple raters assign label for the same and define a requirement for agreement
    - E.g., use four raters and require that at least three raters agree
    - Needs to define what happens with data without agreement (e.g., unlabeled, additional steps to assign label)
  - Multiple raters assign label for the same task, assign label if all agree and require additional meeting to achieve consensus for all others
    - Very strict criterion that should greatly reduce noise
    - Effort is higher than for other strategies, because achieving consensus requires meetings or additional discussions
  - When labels can be mapped to a Likert scale, it may also be possible to calculate a mean value over multiple raters as final label
    - Likert scales are used for rating how much something applies (what you always see when you evaluate lectures)
  - …

Inter-rater Agreement

- How often raters agree with each other important to understand validity of data
- Many agreements mean that the likelihood of noise and bias is reduced
  - (Unless all raters would make the same mistakes and have the same biases)
- Disagreements can have various reasons, e.g.,
  - Difficult task with a high error rate
  - Active disagreement between raters, because they judge data differently
  - Biases affect ratings leading to differences
- Studying where disagreements occur can also be an interesting research subject
  - E.g., for which aspects developers agree and where (and why?) they disagree
- Statistical approaches to measure inter-rater agreement exist
- Note: there is also the related concept of intra-rater agreement, i.e., how robust is the rating by a person
  - In other words: to people actually agree with themselves when solving the same problem multiple times
  - We do not consider the differences between inter- and intra-rater reliability in this lecture

Cohen‘s κ

- Measures the agreement between two raters when assigning k different nominal labels

- Probability of agreement
  - $p_0 = \frac{Number\ of\ agreements}{Number\ of\ total\ ratings}$

- Probability of chance agreement
  - $p_e = \frac{1}{N^2} \sum_{i=1}^{k} n_{i1} n_{i2}$ where
    - N is the number of ratings
    - n_{ki} the number of times rater j assigned category i
- The agreement can then be calculated as
  - $\kappa = \frac{p_0 - p_e}{1 - p_e} = 1 - \frac{1 - p_0}{1 - p_e}$
  - This is the counterprobabilitiy of the probabilities of actual disagreement and chance disagreement

Cohen, Jacob (1960). "A coefficient of agreement for nominal scales". Educational and Psychological Measurement. 20 (1): 37–46.

Interpretation of Cohen's κ

- Usually based on a table from Landis and Koch
  - Not universally accepted!
  - Does not take into account that agreement is usually higher with less categories

κ	Interpretation
≤ 0	Poor agreement
$(0, 0.2]$	Slight agreement
$(0.2, 0.4]$	Fair agreement
$(0.4, 0.6]$	Moderate agreement
$(0.6, 0.8]$	Substantial agreement
$(0.8, 1.0]$	Almost perfect agreement

Landis, J.R.; Koch, G.G. (1977). "The measurement of observer agreement for categorical data". Biometrics. 33 (1): 159–174

Beyond Cohen‘s κ

- Fleiss‘ κ generalizes the concept of Cohen‘s κ to more than two raters

- Krippendorf‘s α generalizes the concept of Cohen‘s κ to any number of raters, incomplete data, and different scales (e.g., ordinal, interval, ratio)

- Cronbach‘s α is a correlation-based approach that can take the difference between ratings into account
  - Takes into account that when ratings from 1-10 are conducted, that 8 is more similar to 7 than to 1.
  - Standard approach when Likert-scales are used

Deductive coding – closed coding

- Works with a fixed code book
- Requires good knowledge about categories before execution of the coding
  - E.g., rate sentiment as happy, sad, or neutral
- Makes definition of the categories part of the study design
  - In the above example, other sentiments, like anger, would be ignored.
- Can be both advantage and drawback
  - Helps to focus coding to specific tasks
  - Limits coding to foreseen values which may lead to overlooking relevant aspects

Inductive coding – open coding

- Raters can add new codes
  - Can start with an initial codebook that is extended or with an empty codebook
- Raters need guidance what sort of codes should be used
  → Key part of the study design, otherwise anything could be result
  - E.g., study design could state that codes related to sentiment should be assigned
- Codes can be flexibly adjusted based on what is within the data
  - E.g., would define suitable sentiment after reading a statement – if the sentiment is not in the codebook, this is extended
- Usually more effort than closed coding
  - Requires reasoning about which codes there could be
  - Requires identification of suitable code works

Harmonizing Open Codes

- Two raters may use different codes to describe the same concept
  - E.g., two raters analyze teaching materials. One describes these materials as *PowerPoint*, the other as *Slides*
- This may lead to problems
  - E.g., underestimation of inter-rater agreement
  - Overlooking aspects in subsequent analysis, because data is in different categories when it should be the same
- Harmonizing codes means that such differences are resolved by replacing codes with a unified representation
  - E.g., replace codeword *PowerPoint* with *Slides*
- Requires discussion between raters
  - Can be done at fixed intervals (e.g., after 20 some percentage of data)
  - Can be done after the fact (i.e., once all codes are available)

- Raters can also start to initialize the codebook together by rating a certain percentage of the data together
  - Pre-fills codebook with entries that will be used consistently, but no independent ratings for these elements

Axial coding

- Process of relating codes to each other
  - Can be interpreted as coding of codes
- Models the relationships between codes
  - Requires a strategy, same as the initial coding
  - Can be used to group codes
- Grouping on the same level of abstraction as alternative to harmonization
  - Would preserve the initial different code works
- Grouping on different levels of abstraction to interpret information
  - E.g., group codes *Slides* and *Coding Exercise* into code *Materials*
  - This is the core concept on which the qualitative analysis methodology grounded theory is build

Grounded Theory Methodology: An Overview. In N. Denzin & Y. Lincoln Handbook of Qualitative Research. 1st ed.

Questions?

